

Unity (C#) - IDE Setup Guide

Companion to the Complete Codebase - Semester 1 Capstone

VIDEO GAME CREATION

The code alone won't run - several fields only exist in the Inspector, and a few GameObjects/prefabs need to be built or confirmed by hand. This document covers every piece of that point-and-click setup, organized by GameObject, so you can work through it top to bottom in a fresh (or your own existing) project. Assumes you're starting from a project that already has Lessons 1-13's objects in place; sections marked NEW are what Lesson 14's five stretch features add.

Scene Hierarchy

The full hierarchy, as it exists in the course's own reference project. Indentation shows parent/child; (NEW) marks objects this deliverable adds on top of what Lesson 13 already had.

```
Main Camera
Global Light 2D
GameManager                          <- GameManager.cs
EnemySpawner                         <- EnemySpawner.cs
Enemies                              (empty parent - organizes spawned enemy
instances, optional but tidy)
Player                               <- PlayerMovement.cs, Player.cs, tag: Player
Canvas
  ScoreText / LivesText / WaveText   (UI Text, read by HUD.cs)
HUD                                  <- HUD.cs (references the three Text objects
above)
EventSystem                          (auto-created by Unity when the first Canvas
is added)
PowerUpSpawner      (NEW)            <- PowerUpSpawner.cs
FormationController (NEW)            <- FormationController.cs
Shield_1, Shield_2, Shield_3 (NEW) <- Shield.cs  (3 separate placed
instances, not spawned)
```

Enemy, Bullet, Explosion, and (NEW) PowerUp are PREFABS, not scene objects - they only exist in the Project window and get Instantiate()'d at runtime. Nothing to place in the hierarchy for them.

Tags (Edit > Project Settings > Tags and Layers, or the Inspector's Tag dropdown)

- "Enemy" - on the Enemy prefab. Read by Bullet.cs's OnTriggerEnter2D and EnemySpawner.cs's ClearEnemies().
- "Player" - on the Player GameObject. Read by (NEW) PowerUp.cs's OnTriggerEnter2D.

GameManager

One component, GameManager.cs. All fields below are public, so they show up directly in the Inspector - nothing needs [SerializeField] guessing.

Field	What to assign
spawner	`Drag the EnemySpawner GameObject here.`
lifeLostSound / gameOverSound / winSound / waveClip	`Your existing Lesson 12 sound effects (sfx_lifelost, sfx_gameover, sfx_win, sfx_wave or equivalent) - unchanged from before.`
gameStartSound	`NEW - picked when the title screen's Space press starts the run. Any short confirmation/start sound works; reusing an existing clip (even sfx_wave) is completely fine if you don't want to source a new one.`

No Inspector field for the title text itself ("Kestrel Watch") or wave total - those are hardcoded in OnGUI()/Start() as literals, matching how the win/lose banner text has always worked since Lesson 10.

EnemySpawner

Field	What to assign
enemyPrefab	`Drag the Enemy prefab here.`

Enemy (prefab)

Existing components: SpriteRenderer, BoxCollider2D (Is Trigger checked - Bullet's OnTriggerEnter2D needs this), tag Enemy, plus Enemy.cs.

Field	What to assign
explosionPrefab	`Drag the Explosion prefab here.`
spriteRenderer	`Drag the Enemy prefab's own SpriteRenderer component here (usually itself - drag the GameObject into the field, Unity will resolve the component).`
flashSprite	`The white hit-flash sprite from Lesson 13 (enemy_invader_white or equivalent).`
normalSprite	`Leave EMPTY - Enemy.cs's own Start() fills this in automatically from whatever sprite is already showing, so hand-assigning it would just get overwritten.`

Bullet (prefab)

Existing components: SpriteRenderer, BoxCollider2D (Is Trigger checked), Rigidbody2D (Body Type: Dynamic, Gravity Scale: 0 - required for Unity's 2D physics to actually fire the trigger event at all; without a Rigidbody2D on at least one side of a trigger pair, OnTriggerEnter2D never fires), AudioSource, plus Bullet.cs.

Field	What to assign
speed	`Default 6 is fine - unchanged from Lesson 3.`
hitSound	`Your existing sfx_hit clip from Lesson 12/13.`

Explosion (prefab)

SpriteRenderer only, plus Explosion.cs. No Inspector fields on the script itself - it's fully self-contained (Lesson 13).

Player

Two separate AudioSource components are needed on this GameObject, not one - Player.cs's shoot sound is a one-shot .Play() triggered by firing, while PlayerMovement.cs's walkingBeep is a continuous loop started/stopped as the player moves. Trying to share one AudioSource between both will cause the shoot sound to cut off the walking beep (or vice versa). Add Component > Audio Source twice, and keep track of which is which when wiring the two scripts below.

IMPORTANT - on BOTH AudioSource components, uncheck Play On Awake (it's checked by default whenever you add one). Player.cs and PlayerMovement.cs only ever call .Play() themselves when they actually want a sound to happen; if Play On Awake is left checked and that AudioSource already has a clip assigned (which it needs, for .Play() to have anything to play), Unity fires it the instant the scene loads - which is exactly what a "shooting sound at startup, before you've even pressed anything" turns out to be. On the shoot-sound AudioSource, also leave Loop unchecked; on the walking-beep one, check Loop (PlayerMovement.cs starts/stops it itself, but a non-looping clip would only ever play once per Play() call instead of continuing for as long as the player keeps moving).

NEW - also add a Collider2D with Is Trigger checked, if Player doesn't already have one. Earlier lessons never needed this (nothing ever detected the Player directly), but (NEW) PowerUp.cs's OnTriggerEnter2D now needs a Collider2D on BOTH sides of the pair to fire at all - PowerUp having one isn't enough by itself. If you're still using the original smiley-face sprite, a CircleCollider2D fits it better than a Box; either works as long as Is Trigger is checked, so it doesn't start physically blocking enemies or bullets.

Player.cs

Field	What to assign
bulletPrefab	`Drag the Bullet prefab here.`
fireCooldownMax	`Default 0.2 - unchanged from Lesson 3.`
bulletYOffset	`Default 0.7 - unchanged (spawns the bullet slightly above the player, not exactly on top of it).`

Field	What to assign
audioSrc	Drag Player's FIRST AudioSource component here (the shoot-sound one). On that AudioSource itself (not a script field - there isn't one for this), set its own Audio Clip to your shoot sound effect, Play On Awake OFF, Loop OFF - see the note above.

PlayerMovement.cs

Field	What to assign
spriteDefault / spriteLeft / spriteRight / spriteUp / spriteDown	Your four directional sprites plus the idle sprite, from Lesson 2's setup.
sr	Drag Player's SpriteRenderer component here.
walkingBeep	Drag Player's SECOND AudioSource component here (the looping footstep one) - NOT the same one as Player.cs's audioSrc above. On that AudioSource itself, set its own Audio Clip to your footstep/walk sound, Play On Awake OFF, Loop ON - see the note above.
xBoundary / yBoundary	Defaults 8 / 4.5 are fine - matches the existing play-area clamp.

HUD

Field	What to assign
scoreText / livesText / waveText	Drag the Canvas's ScoreText / LivesText / WaveText UI Text objects here respectively.

If your own project still has the old debug "GameActive" Text object under Canvas (visible in some earlier playtesting builds), it's safe to delete - HUD.cs no longer has a field for it, and it was never meant to ship.

NEW for Lesson 14's Stretch Features

Everything below is new - none of it exists in a Lesson 13 project. Build these in roughly this order; each is independent of the others, so you can wire one, test it, then move to the next.

1. Destructible Shields

- Create a new sprite GameObject (any simple rectangle/barrier sprite - a placeholder colored square works fine), add a BoxCollider2D with Is Trigger checked, and attach Shield.cs.
- Also add a Rigidbody2D - Body Type: Kinematic. This is easy to miss since the Shield doesn't move: Unity only fires OnTriggerEnter2D when at least one side of the pair has a Rigidbody2D, and the Enemy prefab doesn't have one (it's only ever been the Bullet side of a trigger pair before now, and Bullet already carries its own Rigidbody2D). Without this, enemies pass straight through every shield with no reaction at all. Kinematic keeps the shield from falling or being pushed - it still only reacts because Shield.cs's own OnTriggerEnter2D code runs.
- Duplicate it 2-3 times and place the copies in a row between the enemy spawn area and the Player, with some horizontal gap between them so they don't all block the same lane.
- Name them something you can find later (Shield_1, Shield_2, Shield_3) - they're placed scene objects, not a prefab/spawner, so there's no automatic creation to debug if one looks wrong.

Field	What to assign
hitsRemaining	`Default 3 is fine - how many enemy hits a shield absorbs before it's destroyed.`
spriteRenderer	`Drag the Shield's own SpriteRenderer here.`
damageColors	`IMPORTANT - size this array to (hitsRemaining - 1), not hitsRemaining. With the default hitsRemaining = 3, set Size to 2 and fill in two colors (e.g. a yellow "first hit" color, then a red "about to break" color) - the shield's ORIGINAL color is what shows before any hit lands, so it doesn't need its own slot in this array. Sizing it to 3 instead still basically works, but the color in slot 0 is skipped over entirely (the math always lands on slot 1 or later) - so if you want every color you add to actually get used, size it to (hitsRemaining - 1).`

2. Power-Up

- Build a new prefab: a small pickup sprite with a BoxCollider2D (Is Trigger checked) and PowerUp.cs attached. Save it to your Prefabs folder as PowerUp.
- Also add a Rigidbody2D to this prefab - Body Type: Kinematic. Same reason as the Shield above: neither PowerUp nor Player has one otherwise, so OnTriggerEnter2D never fires and the pickup can't be collected. Kinematic won't fight PowerUp.cs's own Translate()-based fall.
- Create a new empty GameObject in the scene named PowerUpSpawner, and attach PowerUpSpawner.cs to it.

Field	What to assign
powerUpPrefab (on PowerUpSpawner)	`Drag the new PowerUp prefab here.`
minInterval / maxInterval (on PowerUpSpawner)	`Defaults 8 / 15 (seconds) are fine - how often a power-up drops.`
fallSpeed (on the PowerUp prefab itself)	`Default 1 is fine.`

No wiring needed on Player.cs for this - PowerUp.cs calls player.ActivateRapidFire(5f) directly once it detects the Player's tag, and that method already exists on Player.cs.

3. Bonus Enemy

No new GameObject or prefab at all - this one is pure code, already wired into EnemySpawner.cs (the last enemy slot on every even-numbered wave gets extra points, extra speed, and a gold tint automatically). Nothing to set up in the Inspector.

4. Formation-Marching Movement

- Create a new empty GameObject in the scene named FormationController, and attach FormationController.cs to it.

Field	What to assign
moveSpeed / stepDown / xBoundary	`Defaults (1.5 / 0.5 / 8) are fine to start - tune by feel once you see wave 2 running. Don't drop xBoundary below about 6.5: wave 2 spawns its 6 enemies spread across x = -5 to +5 already, so a smaller boundary (4, say) has enemies starting past the edge and stepping down every single frame from the moment the wave spawns - the formation will look like it's free-falling instead of marching. 6.5 is the bare minimum that avoids that outright, but only gives about a second of sideways travel before the first step down, which reads as a fast pass immediately followed by a hard drop; 8 gives a few extra seconds of real marching first and is the better starting point to tune from.`

Nothing to wire on Enemy.cs or EnemySpawner - FormationController finds every live Enemy itself each frame (FindObjectsByType<Enemy>()), and Enemy.cs already checks FormationController.formationModeActive on its own.

5. Title / Game-Over Screen

No new GameObject - this is GameManager.cs's own OnGUI(), the same mechanism that already draws the win/lose banner. Nothing to set up beyond GameManager's gameStartSound field above. Just confirm GameManager itself is still in the scene and active.

Known Gotchas

- If nothing spawns and the screen just sits on "Press SPACE to begin your watch" forever: confirm GameManager's own GameObject is active and that Unity's new Input System package is installed/enabled (Keyboard.current is part of the new Input System, not the legacy Input class) - this has been required since early lessons, but worth the first thing to check if the title screen seems unresponsive.

- If the shoot sound silences the walking beep (or vice versa): you likely wired both Player.cs's audioSrc and PlayerMovement.cs's walkingBeep to the SAME AudioSource component. Add a second AudioSource and split them - see the Player section above.
- If formation-marching enemies never appear: remember it's wave-PARITY-based (even waves only) - wave 1 is lane descent by design. Let a run reach wave 2 before concluding it's broken.
- If wave 2's formation plummets to the bottom almost instantly instead of marching side to side: xBoundary on FormationController is too small for that wave's spawn spread - see the Formation-Marching section above.
- If enemies pass straight through a shield with no reaction: the shield is missing its Rigidbody2D (Kinematic) - see the Shields section above.
- If a shield's first damage color never seems to show up: check damageColors is sized to (hitsRemaining - 1), not hitsRemaining - see the Shields section above.
- If a power-up falls through the Player without being collected: check both sides of the pair - PowerUp needs its Rigidbody2D (Kinematic), AND Player needs a Collider2D (Is Trigger checked) of its own - see the Power-Up and Player sections above.
- If a wave gets stuck even though no enemies are visible on screen: this is the Lesson 10/11 stalemate bug resurfacing - something destroyed an enemy without telling GameManager. Shield.cs's own code already calls GameManager.RegisterKill(0) for this reason; if you've since customized any of the new scripts, make sure every path that destroys an enemy GameObject also calls a GameManager Register method.
- If a wave (or the whole game) clears early with an enemy still visibly on screen: this is the same bug's opposite twin - a Bullet and a Shield reaching the same enemy in the same instant could each register their own kill on it, double-counting one enemy as two. Bullet.cs and Shield.cs both guard against this with an other.enabled check now; nothing to set up, just something to know if you've since modified either script.
- Shields reset themselves automatically on Restart (R) - GameManager.ResetGame() finds every Shield in the scene, including ones already used up, and restores each to its starting hitsRemaining, color, and visibility. Nothing to wire for this; it's built into Shield.cs's own ResetShield() method.
- If you hear a shooting sound the instant the scene loads, before the title screen's even been dismissed: an AudioSource on Player still has Play On Awake checked. See the IMPORTANT note under Player above.
- If the very Space press that starts (or restarts) the game also fires a bullet: this was a real bug, not a setup gap - fixed in Player.cs with a frame-count check against GameManager.startFrame. If you've hand-edited Player.cs's fire condition, make sure that check is still there.

Fine-Tuning Reference

Where every speed, timing, and difficulty-curve number in this project actually lives, and whether you change it in the Inspector or in the script. Inspector values are per-instance (a prefab's value applies to every copy unless overridden); code values apply everywhere at once and need a re-save/re-test, but there's nothing wrong with opening a script and changing a number - that's expected tuning, not "serious" editing.

Value	Where	What it affects
Bullet.speed	`Inspector - Bullet prefab`	How fast shots travel up the screen.
Bullet.cs's flash duration (0.1f) and flash scale (1.3f)	`Code - Bullet.cs, Start()`	The brief yellow, slightly-enlarged muzzle-flash look right as a bullet spawns, fading back to normal after 0.1 seconds. Raise the 0.1 for a longer flash, or the 1.3 for a bigger one.
Player.fireCooldownMax	`Inspector - Player`	Time between shots. The power-up temporarily overrides this to 0.05 (hardcoded in Player.cs's RapidFireRoutine) - change that line if you want rapid-fire to feel different, not this field.
PlayerMovement's private speed field (3f)	`Code - PlayerMovement.cs`	Player's own movement speed. Not a [SerializeField], so it's not in the Inspector - edit the line directly.
Enemy.speed starting value (1.5f)	`Inspector - Enemy prefab`	Only matters for the very first enemy spawned before EnemySpawner overwrites it on every spawn - see the next row. Safe to ignore.
EnemySpawner's per-wave speed formula (1.5f + (wave-1)*0.5f)	`Code - EnemySpawner.cs, SpawnEnemies()`	The REAL enemy speed for every wave - this overwrites Enemy.speed right after each enemy spawns. Change the 1.5f (base) or 0.5f (per-wave increase) to adjust overall difficulty ramp.
Enemy.cs's Random.Range(0.75f, 1.3f) speed variation	`Code - Enemy.cs, Start()`	Multiplies the wave-based speed above by a random amount per enemy, so a wave isn't one perfectly uniform wall of movement. Lane descent only - skipped during formation mode, since a formation member's own speed field never gets read while FormationController is the one moving it. Widen or narrow the range to make waves feel more or less chaotic.
EnemySpawner's per-wave count formula (4 + wave)	`Code - EnemySpawner.cs, SpawnEnemies() and GameManager.cs (wave resets)`	How many enemies spawn each wave. This also drives the spawn-position spread (see FormationController's xBoundary note below) - widening it means xBoundary needs to grow too.
GameManager.totalWaves (3)	`Code - GameManager.cs`	A static field, so it won't appear in the Inspector even though it looks like a normal number - how many waves before the game is won.
FormationController.moveSpeed	`Inspector - FormationController`	How fast formation-mode enemies march side to side (wave 2, and any later even wave if you raise totalWaves).
FormationController.stepDown	`Inspector - FormationController`	How far the whole formation drops each time it hits an edge.

Value	Where	What it affects
FormationController.xBoundary	`Inspector - FormationController`	How wide the marching lane is. Must stay noticeably wider than that wave's spawn spread (half-width = 3 + wave) or the formation free-falls instead of marching - see the Formation-Marching section above for the full explanation. If you raise totalWaves past 3, re-check this for the new highest even wave.
Shield.hitsRemaining	`Inspector - each Shield instance`	How many enemy hits that particular shield absorbs before breaking. Can differ shield to shield since these are hand-placed, not a shared prefab.
PowerUp.fallSpeed	`Inspector - PowerUp prefab`	How fast a dropped power-up falls toward the Player.
PowerUpSpawner.minInterval / maxInterval	`Inspector - PowerUpSpawner`	How often a power-up drops (randomized between the two).
PowerUp's rapid-fire duration (5f)	`Code - PowerUp.cs, OnTriggerEnter2D's ActivateRapidFire(5f) call`	How long the power-up's effect lasts after pickup.
EnemySpawner's score-variety range (Random.Range(5, 16))	`Code - EnemySpawner.cs, SpawnEnemies()`	Per-enemy point value for normal (non-bonus) kills - Lesson 9's Level 3 scoring variety.
OnGUI's fade-in duration (0.5f)	`Code - GameManager.cs, OnGUI()`	How long the win/lose text takes to fade in - Lesson 10's Level 3 animation option.